

Módulo 5 – Capítulo 07 – Sección 01

Métricas de evaluación: exactitud, fidelidad, relevancia y coherencia

La evaluación de un sistema de IA exige responder preguntas que no tienen respuesta binaria: ¿esta respuesta es correcta? ¿es suficientemente completa? ¿está fundamentada en el contexto provisto o inventa información? Estas preguntas corresponden a dimensiones diferentes de la calidad que ninguna métrica única puede capturar. La decisión de qué métricas usar no es técnica en el sentido estricto sino estratégica: depende de qué fallos del sistema serían más costosos para el negocio, qué dimensiones de la calidad importan más a los usuarios, y qué aspectos del comportamiento el equipo necesita monitorizar para mejorar el sistema de forma deliberada.

La fidelidad —faithfulness en la literatura de evaluación RAG— mide si cada afirmación de la respuesta puede inferirse del contexto provisto al modelo, sin introducir información que no estaba disponible. Es la métrica más crítica en sistemas RAG donde el riesgo de alucinación es el principal factor de desconfianza del usuario: un sistema de soporte médico que inventa síntomas no mencionados en el documento recuperado, o un asistente legal que cita jurisprudencia que no estaba en el contexto, tiene una fidelidad baja con consecuencias potencialmente graves. RAGAS implementa faithfulness descomponiendo la respuesta en afirmaciones atómicas —frases independientes que afirman algo sobre el mundo— y verificando qué proporción de ellas puede inferirse del contexto recuperado usando el LLM como verificador: `faithfulness = afirmaciones_soportadas / total_afirmaciones`.

La relevancia —answer relevancy— mide si la respuesta responde directamente a la pregunta formulada, penalizando respuestas que aunque sean verídicas resultan tangenciales o que responden una versión diferente de la pregunta. Una respuesta que contesta "¿cuáles son los beneficios de X?" cuando la pregunta era "¿cuáles son los riesgos de X?" puede tener faithfulness perfecta —todo lo que dice está en el contexto— pero relevancy nula. RAGAS mide esta dimensión con una técnica de reverse engineering: genera N versiones de la pregunta original a partir de la respuesta, y mide la similitud coseno entre esas preguntas

generadas y la pregunta real; si la respuesta habla de algo diferente a lo que se preguntó, las preguntas generadas divergen de la pregunta original.

La coherencia evalúa si la respuesta es internamente consistente, gramaticalmente correcta y fluye de forma lógica sin contradicciones entre sus partes. Es especialmente importante en respuestas largas —resúmenes de documentos extensos, análisis de múltiples fuentes— donde el modelo puede contradecirse entre párrafos o repetir información de formas que crean inconsistencia aparente. La exactitud o correctness compara la respuesta con una ground truth conocida, aplicable cuando existe una respuesta objetivamente correcta: preguntas factuales con respuesta única, clasificación con categorías definidas, extracción de entidades con valores esperados. Las métricas de clasificación estándar —accuracy, precision, recall, F1 por clase— son la herramienta correcta para sistemas con salidas discretas y una ground truth verificable.

Aspectos técnicos de las métricas de evaluación

- **Faithfulness en RAGAS:** $\text{faithfulness} = \text{afirmaciones_soportadas} / \text{total_afirmaciones}$; score de 0 a 1; valores inferiores a 0.7 son señal de alucinación sistemática; la descomposición en afirmaciones atómicas hace el score interpretable: se puede ver qué afirmaciones específicas no tienen soporte en el contexto.
- **Answer relevancy en RAGAS:** similitud coseno entre las preguntas generadas por reverse engineering y la pregunta original; valores inferiores a 0.7 indican que el sistema tiende a responder preguntas adyacentes en lugar de la pregunta formulada; útil para detectar prompts que "derivan" temáticamente.
- **Context recall:** mide qué fracción de la información necesaria para responder correctamente aparece en el contexto recuperado; detecta fallos del retriever —no del synthesizer— cuando el sistema falla en recuperar los documentos relevantes.
- **BERTScore F1:** precision (tokens de la respuesta generada presentes en la referencia) y recall (tokens de la referencia presentes en la respuesta) a nivel de embeddings contextuales de BERT; más robusto que BLEU ante variación léxica en dominios con vocabulario rico.
- **Exactitud en clasificación:** cuando el sistema clasifica intenciones o categorías, la matriz de confusión revela qué clases se confunden sistemáticamente (ej. "cancelación" vs "modificación" de un pedido), orientando la mejora de prompts hacia las clases con menor F1.

El sistema de evaluación no es un endpoint final del proceso de desarrollo sino un instrumento de navegación: las métricas informan qué dimensión del sistema necesita mejora, en qué subgrupos de usuarios hay problemas específicos, y si los cambios aplicados están produciendo el efecto esperado. La siguiente sección examina los frameworks que automatizan el cálculo de estas métricas sobre datasets y las integran en pipelines de CI.

Para recordar: Un sistema evaluado únicamente con faithfulness puede tener respuestas fieles pero irrelevantes; únicamente con relevancy puede tener respuestas relevantes pero alucinadas; la evaluación robusta requiere al menos tres métricas complementarias que cubran fidelidad, relevancia y exactitud para diferentes tipos de consulta.

Módulo 5 – Capítulo 07 – Sección 02

Frameworks de evaluación: RAGAS, DeepEval y TruLens

La instrumentación manual de métricas de evaluación —calcular faithfulness, answer relevancy y context recall para cada caso de un dataset de 200 ejemplos, agregar los resultados, y comparar con el baseline— es un proceso que puede automatizarse completamente. Los frameworks de evaluación de IA hacen exactamente eso: proveen una API donde el dataset de evaluación se describe como una estructura de datos estándar, el cálculo de métricas se ejecuta con una llamada, y los resultados se presentan en formatos integrables con pipelines de CI. La elección entre los frameworks disponibles depende del tipo de sistema que se evalúa, de cómo el equipo prefiere consumir los resultados, y del modelo evaluador disponible para el presupuesto del proyecto.

RAGAS es el framework de referencia para la evaluación de sistemas RAG. Su API acepta un dataset de HuggingFace con columnas `question`, `answer`, `contexts` (lista de fragmentos recuperados), y opcionalmente `ground_truth` (la respuesta de referencia para métricas que la requieren). La función `evaluate(dataset, metrics=[faithfulness, answer_relevancy, context_recall])` devuelve un DataFrame con los scores por caso y las métricas agregadas. La integración directa con LangChain y LlamaIndex — `from ragas.integrations.langchain import EvaluatorChain` — captura automáticamente los contextos recuperados en cada query sin necesidad de instrumentar el código manualmente. El modelo evaluador interno —que RAGAS usa para calcular faithfulness y answer relevancy— puede configurarse con cualquier proveedor soportado por LangChain; usar un modelo más capaz que el evaluado mejora la confiabilidad pero aumenta el costo.

DeepEval ofrece un framework de evaluación más general, no específico de RAG, con una API compatible con pytest: los tests se escriben con `LLMTestCase`, se decoran con `@pytest.mark.deepeval`, y se ejecutan con `pytest`, produciendo output compatible con JUnit XML para integración directa en GitHub Actions sin infraestructura adicional. `AnswerRelevancyMetric(threshold=0.7)`, `FaithfulnessMetric(threshold=0.8)`, `HallucinationMetric(threshold=0.5)`, `ToxicityMetric(threshold=0.1)` son ejemplos de métricas configurables con umbrales de paso/fallo. La ventaja de DeepEval sobre RAGAS es

su mayor variedad de métricas —incluyendo seguridad como toxicidad y bias— y su integración nativa con el flujo de testing de pytest; la desventaja es menor integración directa con LlamaIndex y LangChain.

TruLens añade una interfaz visual de dashboard web que permite explorar las trazas de evaluación, comparar experimentos y analizar distribuciones de scores a lo largo del tiempo, siendo especialmente útil para equipos que necesitan una interfaz visual en lugar de resultados en consola o CI. Los wrappers `TruChain` y `TruLlama` instrumentan LangChain y LlamaIndex respectivamente, capturando automáticamente la traza completa de cada query —recuperación, síntesis, herramientas— y calculando el "RAG triad" de TruLens: context relevance (¿el contexto es relevante para la pregunta?), groundedness (¿la respuesta está fundamentada en el contexto?) y answer relevance (¿la respuesta responde la pregunta?). `tru.get_leaderboard()` genera una tabla comparativa de experimentos con scores medios, facilitando la selección entre versiones de prompts o modelos.

Componentes principales de los frameworks de evaluación

- **RAGAS** `evaluate()` : recibe un `Dataset` de HuggingFace con columnas `question`, `answer`, `contexts` y devuelve un dict con el score medio de cada métrica; análisis por caso disponible en el DataFrame resultante para identificar qué casos específicos fallan.
- **DeepEval** `@pytest.mark.deepeval` : decorador que ejecuta métricas sobre el `LLMTestCase` y genera reportes en `deepeval_results/`; el threshold de cada métrica es configurable por test; output JUnit XML para integración con CI sin configuración adicional.
- **TruLens** `TruChain` y `TruLlama` : wrappers que capturan automáticamente retrieval + generation como traza completa; `tru.get_leaderboard()` compara experimentos con visualización del RAG triad; accesible via dashboard web en `localhost:8501`.
- **Selección del modelo evaluador:** usar un modelo más capaz que el evaluado para el juez interno mejora la confiabilidad de las métricas; calcular el costo esperado del modelo evaluador antes de seleccionarlo (`dataset_size × avg_tokens_per_case × evaluador_precio`).
- **Costo de la evaluación automatizada:** para datasets de 100 a 500 casos el costo típico con modelos de evaluación eficientes es de \$1 a \$10 por run; calcularlo antes de configurar el scheduled job de CI para evitar sorpresas de facturación.

Un mapa unificado de herramientas de evaluación: RAGAS y DeepEval son las opciones principales para evaluación integrada en CI por su compatibilidad con pytest y JUnit XML;

TruLens es la opción para equipos que priorizan exploración visual de trazas; LangSmith Datasets combina dataset management, evaluación y registro histórico en una sola plataforma. La elección no es excluyente: RAGAS para las métricas de RAG en CI y TruLens para la exploración exploratoria son combinaciones válidas.

Principio rector: La elección del framework de evaluación debe basarse en el tipo de sistema (RAG, agente, clasificador), el canal de consumo de resultados (CLI en CI, dashboard visual, integración pytest), y el modelo evaluador disponible en el presupuesto; más de un framework puede usarse para propósitos diferentes en el mismo sistema.

Módulo 5 – Capítulo 07 – Sección 03

Evaluación con LLM-as-Judge: diseño de jueces y criterios

El LLM-as-Judge es el patrón de evaluación que usa un modelo de lenguaje para evaluar la calidad de las respuestas de otro sistema, reemplazando o complementando la evaluación humana en casos donde el volumen de datos hace la anotación manual inviable. Un sistema que genera 10.000 respuestas por día no puede ser evaluado manualmente para cada respuesta; un juez LLM puede evaluar ese mismo volumen con un costo de pocos dólares en minutos. El patrón escala donde la evaluación humana no puede, pero produce resultados confiables solo cuando el diseño del juez es riguroso: el juez LLM con un prompt mal diseñado no es evaluación automatizada, es ruido automatizado.

La confiabilidad del juez LLM depende casi enteramente del diseño del prompt de evaluación. Un prompt genérico como "¿Es esta respuesta buena? Responde con 1 a 5" produce evaluaciones con alta varianza, sesgos sistemáticos y baja correlación con el juicio humano. Un prompt con rúbrica explícita, ejemplos concretos para cada nivel de la escala, y criterios separados para cada dimensión de calidad produce evaluaciones que correlacionan con el juicio humano con $r > 0.8$ en benchmarks como MT-bench. La diferencia entre ambos prompts no es estética sino funcional: el primero delega la definición de "buena" al modelo evaluador —que interpretará ese término de forma inconsistente—; el segundo le provee una definición operacional específica que puede aplicar de forma consistente.

Los criterios de evaluación deben definirse por separado para cada dimensión de calidad, evitando el "halo effect" donde la evaluación holística está dominada por la primera impresión. Un prompt que primero evalúa completitud —"¿La respuesta aborda todos los aspectos explícitamente preguntados?"—, luego precisión factual —"¿Las afirmaciones son consistentes con el contexto provisto?"—, y finalmente formato —"¿La respuesta cumple el formato especificado en las instrucciones?"— produce puntuaciones por dimensión que son más informativas y más consistentes que una puntuación holística única. El chain-of-thought en el juez refuerza la consistencia: pedir al modelo que explique su razonamiento antes de asignar el score —"Primero analiza cada criterio, luego asigna la puntuación"— hace el

proceso de evaluación más transparente y permite revisar el razonamiento cuando el score parece incorrecto.

Los sesgos conocidos del LLM-as-judge requieren mitigaciones explícitas. El position bias —el modelo favorece la primera o última opción en comparaciones A/B— se mitiga ejecutando la evaluación dos veces con el orden de las opciones invertido y promediando; si el resultado cambia significativamente al cambiar el orden, la diferencia de calidad no es estadísticamente significativa. El verbosity bias —el modelo favorece respuestas más largas independientemente de su calidad— se mitiga con instrucciones explícitas en la rúbrica: "No penalices la concisión; una respuesta corta y completa es mejor que una larga y redundante." El self-enhancement bias —Claude evaluador puede favorecer respuestas de Claude— se mitiga usando modelos evaluadores de diferentes proveedores o verificando la calibración del juez con datos humanos.

Aspectos técnicos del diseño de jueces LLM

- **Rúbrica con ejemplos ancla:** definir cada nivel de la escala con un ejemplo concreto del dominio específico ("Nivel 1: la respuesta no menciona el tema de la pregunta. Ejemplo: [ejemplo real del dominio]"); los descriptores abstractos ("malo", "regular", "bueno") producen evaluaciones inconsistentes.
- **Evaluación multi-criterio:** pedir al juez que evalúe completitud, precisión factual y formato por separado antes de dar una puntuación global; las puntuaciones por criterio son más informativas que la global y revelan en qué dimensión específica está el problema.
- **Mitigación del position bias:** en comparaciones A/B, ejecutar la evaluación con orden A→B y luego B→A; si el juicio cambia al cambiar el orden, la diferencia de calidad no es robusta y requiere más evidencia para tomar una decisión.
- **Chain-of-thought en el juez:** "Primero analiza los criterios siguiendo la rúbrica. Luego, en una nueva línea, asigna el score en el formato: SCORE: N" mejora la consistencia y hace el razonamiento auditable; el texto del razonamiento puede revisarse cuando el score es inesperado.
- **Calibración con datos humanos:** comparar los scores del juez LLM con una muestra de 50 a 100 evaluaciones humanas; una correlación de Spearman inferior a 0.7 indica que el juez necesita rediseño del prompt antes de ser confiable para tomar decisiones de despliegue.

Nota del Arquitecto: El error más frecuente en la implementación de LLM-as-judge es usarlo sin calibración contra datos humanos del dominio específico, asumiendo que porque el modelo es "inteligente" sus evaluaciones son correctas. En la práctica, los sesgos del modelo evaluador —*verbosity*, *self-enhancement*, *sensibilidad al orden*— pueden hacer que el juez apruebe sistemáticamente cambios que degradan la calidad percibida por los usuarios. Invertir dos horas en calibrar el juez con 50 evaluaciones humanas es la práctica más importante antes de integrarlo en el pipeline de CI.

Documentar el prompt exacto del juez LLM y la versión del modelo evaluador como parte del sistema de evaluación versionado — `eval/judge_prompt_v1.2.txt` , `eval/judge_config.yaml: {model: claude-3-5-sonnet-20241022}` — garantiza que los scores históricos son comparables: un cambio en cualquiera de los dos puede cambiar sistemáticamente los scores haciéndolos incomparables con el baseline.

Buena práctica: Documentar el prompt exacto del juez LLM y la versión del modelo evaluador como artefactos versionados del sistema de evaluación, ya que cambios en cualquiera de los dos pueden cambiar los scores históricos haciendo imposible la comparación temporal y el tracking de la mejora del sistema.

Módulo 5 – Capítulo 07 – Sección 04

Benchmarks comparativos: modelos, versiones y configuraciones

La decisión de qué modelo de LLM usar para una tarea específica raramente puede tomarse con confianza basándose en rankings generales como MMLU, HumanEval o Chatbot Arena: estas evaluaciones miden capacidades en dominios generales que pueden no correlacionar con el rendimiento en el dominio específico del sistema. Un modelo que encabeza los rankings de razonamiento matemático puede no ser el mejor para clasificar intenciones de soporte técnico en español con terminología de telecomunicaciones; solo un benchmark interno sobre el dominio específico puede responder esa pregunta con evidencia empírica.

El proceso de benchmarking interno sigue cuatro pasos que deben ejecutarse de forma controlada para producir resultados comparables. El primero es definir el conjunto de evaluación: un dataset de 100 a 500 casos representativos del dominio, con criterios de calidad claros para cada caso, construido con los mismos principios de representatividad y cobertura de casos borde que el dataset de testing. El segundo es definir las métricas: las mismas que se usan en producción para que el resultado del benchmark prediga el rendimiento real. El tercero es ejecutar el mismo dataset sobre cada candidato de forma controlada —misma temperatura, mismos seeds donde aplique, misma versión de prompt— para que las diferencias observadas sean atribuibles solo al modelo. El cuarto es comparar con análisis estadístico: calcular intervalos de confianza del 95% para las métricas de cada candidato usando bootstrap resampling, y determinar si las diferencias son estadísticamente significativas o dentro del ruido de muestreo.

Las comparaciones deben ser multidimensionales porque ningún modelo domina en todas las dimensiones relevantes. Un modelo puede tener faithfulness superior pero latencia P95 tres veces mayor y costo por token cuatro veces más alto; la decisión final requiere ponderar estas dimensiones según los requisitos del caso de uso. Un sistema de chat en tiempo real ponderará la latencia con mucho más peso que un pipeline de análisis nocturno; un sistema de soporte médico priorizará la faithfulness sobre cualquier consideración de costo. Esta

ponderación hace que el benchmark no produzca un "ganador" universal sino un ranking específico para el conjunto de requisitos del proyecto.

El análisis por subgrupo es una de las prácticas más valiosas del benchmarking que frecuentemente se omite por la presión de obtener un resultado rápido. Desagregar los resultados por categoría de consulta —preguntas simples de un paso vs preguntas complejas de múltiples pasos, consultas en español vs en inglés, preguntas sobre el dominio A vs el dominio B— puede revelar que el modelo A es superior en promedio pero el modelo B es significativamente mejor en el subgrupo más frecuente de consultas reales. El subgrupo que determina la decisión es el subgrupo más importante para el negocio, no el que produce el score más alto en el promedio global.

Aspectos técnicos del benchmarking comparativo

- **Grid de configuraciones:** matriz de experimentos `{model} × {temperature} × {prompt_version}`; cada combinación se ejecuta sobre el mismo dataset; los resultados se registran en una tabla estructurada que permite filtrar, ordenar y comparar cualquier dimensión.
- **Bootstrap confidence intervals:** calcular IC del 95% para las métricas de cada modelo con `scipy.stats.bootstrap`; dos modelos con scores medios similares cuyos IC se solapan no tienen diferencia estadísticamente significativa; evitar declarar un "ganador" sin este análisis.
- **Latencia y costo como métricas de primera clase:** registrar P50/P90/P99 de latencia y costo por request junto con las métricas de calidad; una diferencia de calidad del 2% rara vez justifica un incremento de costo del 300% o de latencia P99 del 500%.
- **Análisis por subgrupo:** desagregar los resultados por categoría de consulta, longitud del input, idioma, y cualquier otra dimensión relevante del dominio; los subgrupos revelan dónde cada modelo tiene ventajas específicas que el promedio global oculta.
- **Versionado del benchmark:** guardar la configuración exacta, el hash del dataset y los scores por caso de cada run de benchmark en un registro persistente; permite reproducir el benchmark histórico y comparar ante cambios de versión del modelo del proveedor que ocurren sin previo aviso.

El benchmark interno sobre el dominio específico es siempre más predictivo del rendimiento en producción que los benchmarks públicos generales. Construirlo es una inversión de ingeniería que se recupera en la primera decisión de selección de modelo que toma con evidencia en lugar de con intuición.

Para recordar: El benchmark interno sobre el dominio específico es siempre más predictivo del rendimiento en producción que los rankings públicos generales; construirlo con análisis estadístico, métricas multidimensionales y análisis por subgrupo es una de las inversiones de ingeniería de mayor retorno en proyectos de IA de larga duración.

Módulo 5 – Capítulo 07 – Sección 05

Evaluación offline vs evaluación online en producción

La evaluación offline y la evaluación online son complementarias, no alternativas. La evaluación offline —ejecutar el sistema de evaluación sobre un dataset estático curado antes del despliegue— provee control, reproducibilidad y bajo riesgo: el mismo dataset se puede ejecutar múltiples veces con diferentes configuraciones, los resultados son comparables entre runs, y ningún usuario real está expuesto durante el proceso. La evaluación online —evaluar el sistema sobre el tráfico real de producción— provee autenticidad: captura la distribución real de queries del usuario, detecta patrones de fallo que el dataset curado no anticipó, y produce evidencia del comportamiento real del sistema en su estado operacional.

La ilusión de calidad más peligrosa en AI Engineering es un sistema con excelentes métricas offline y comportamiento degradado en producción. Esto ocurre cuando el dataset offline no es representativo del tráfico real: el equipo seleccionó casos "típicos" según su intuición, pero los usuarios reales formulan preguntas de formas que el equipo no anticipó, con vocabulario diferente, con contextos diferentes, o combinando múltiples intenciones en una sola consulta. La evaluación online es el único mecanismo que revela estos gaps entre el dataset curado y la realidad.

La evaluación online puede implementarse con diferentes niveles de intensidad según el presupuesto de costo y latencia disponible. La evaluación exhaustiva evalúa el 100% del tráfico con métricas automáticas ligeras —score de longitud, detección de patrones de alucinación con reglas simples, clasificador de calidad de bajo costo— que no requieren una llamada adicional al LLM evaluador. La evaluación por muestreo evalúa el 1 al 5% del tráfico con métricas más costosas como LLM-as-judge, que requieren una llamada adicional al modelo; el muestreo estratificado por categoría de consulta asegura representatividad de los subgrupos importantes. Las señales de feedback implícito del usuario son métricas de evaluación online gratuitas: la tasa de regeneración de la misma respuesta, el tiempo hasta la siguiente interacción, la tasa de abandono de sesión inmediatamente después de la respuesta, y el feedback explícito de thumbs-up/down son señales valiosas que no requieren ninguna llamada adicional al LLM.

La implementación de la evaluación online asíncrona —sin añadir latencia al request del usuario— sigue el patrón de pipeline de procesamiento en background: los requests y respuestas se loglean como eventos en una cola (SQS, Pub/Sub, o directamente en S3 como logs estructurados), y un job en background consume esos eventos, ejecuta la evaluación sobre la muestra seleccionada, y publica las métricas resultantes al sistema de monitorización. Este proceso puede tomar de segundos a minutos dependiendo del volumen y del modelo evaluador, pero el usuario del sistema no lo percibe porque ocurre completamente fuera del path crítico del request.

Aspectos técnicos de la evaluación online

- **Muestreo estratificado:** seleccionar el X% del tráfico con `random.random() < SAMPLE_RATE`, pero con sobresampling de categorías de baja frecuencia que son de alto riesgo (preguntas médicas, transacciones financieras, decisiones legales); el estratificado asegura que los subgrupos críticos tienen representación suficiente.
- **Pipeline asíncrono con cola:** cada request de producción emite un evento `{request_id, user_input, llm_response, context, metadata}` a SQS o Pub/Sub; el consumidor del queue ejecuta la evaluación y publica el score a Prometheus y al sistema de alertas.
- **Señales de feedback implícito:** loggear `session_id`, `turn_number`, `time_to_next_message`, `thumbs_up_down`, `is_regeneration`, `session_abandoned_after_response`; estas métricas de comportamiento del usuario son los indicadores de calidad más económicos y más directamente ligados al valor del sistema.
- **Dashboard de evaluación online:** panel de métricas en tiempo real con ventana deslizante de 24h y 7d, coloración de degradación cuando el score cae más del umbral respecto al baseline, visible en el canal de operaciones del equipo con alertas automáticas.
- **Feedback loop offline:** los casos con evaluación negativa —score bajo del LLM-as-judge, feedback negativo explícito del usuario, señales de abandono de sesión— se añaden al dataset offline de evaluación como casos nuevos, haciendo que el dataset evolucione con los fallos reales del sistema.

La evaluación offline sin evaluación online es una ilusión de calidad: el dataset offline siempre es una aproximación de la distribución real del tráfico de producción, y los casos donde el sistema falla en producción son sistemáticamente diferentes de los casos del dataset curado porque reflejan la creatividad y la variabilidad real de los usuarios.

Principio rector: La evaluación offline sin evaluación online es una ilusión de calidad: el dataset offline siempre es una aproximación de la distribución real del tráfico de producción; solo la evaluación online captura la diversidad real de las consultas del usuario y detecta los fallos que el dataset curado no anticipó.

Módulo 5 – Capítulo 07 – Sección 06

Cierre: construir un sistema de evaluación continua

Un sistema de evaluación continua no es una herramienta que se ejecuta puntualmente antes de un despliegue y luego se olvida: es un componente permanente del sistema de IA que opera en paralelo con el servicio de producción, evalúa el tráfico real con métricas automáticas, detecta degradaciones antes de que los usuarios las reporten, alimenta el dataset offline con nuevos casos de fallo, y produce métricas visibles para el equipo en tiempo real. Esta permanencia lo distingue de cualquier evaluación puntual: el sistema de evaluación continua es una inversión en el conocimiento del comportamiento del propio sistema, no una verificación de calidad de momento.

La construcción del sistema de evaluación continua debe seguir el mismo principio de incrementalidad que aplica al sistema de IA que evalúa: comenzar con lo mínimo viable y añadir complejidad solo cuando la evidencia demuestra que el incremento de cobertura detecta problemas reales que el sistema actual no detecta. La versión mínima viable es un script de Python que ejecuta RAGAS o DeepEval sobre un dataset de 50 casos en un job de CI, con output de métricas en el log del pipeline. Esta versión mínima ya aporta valor: hace visible la calidad del sistema en cada PR relevante y bloquea merges que degradan las métricas. La complejidad adicional —evaluación online, LLM-as-judge con rúbrica elaborada, feedback loop automático, dashboard de tendencias— se añade cuando el sistema tiene suficiente tráfico real para que la evaluación online sea significativa estadísticamente, y cuando el equipo tiene evidencia de que los fallos en producción no están siendo capturados por el dataset offline actual.

El retorno de inversión de un sistema de evaluación continua es alto y asimétrico: el costo de detectar y corregir una degradación antes del despliegue —horas de trabajo de un ingeniero— es entre diez y cien veces menor que el costo de detectarla en producción, considerando el costo de comunicación con los usuarios afectados, el daño reputacional medible en churn, el tiempo de diagnóstico durante el incidente cuando la presión operacional es alta, y el time-to-fix que en un sistema sin observabilidad puede ser de días. Este multiplicador de costo se conoce de la ingeniería de software tradicional —el principio de que encontrar bugs en

producción es 10x más caro que encontrarlos en test— pero en sistemas de IA tiene una dimensión adicional: las degradaciones de calidad son silenciosas, por lo que sin evaluación continua pueden permanecer sin detectarse durante días o semanas.

El equipo que construye el sistema de evaluación también construye el entendimiento profundo del comportamiento del sistema de IA: sabe qué dimensiones de la calidad son más sensibles a cambios de prompt, en qué categorías de consulta el sistema es más débil, cómo se correlaciona el score de faithfulness con el feedback del usuario, y cuánto margen de mejora queda en cada dimensión. Este conocimiento es un activo de equipo que va más allá de las métricas numéricas: es la diferencia entre un equipo que mejora el sistema con intuición y uno que lo mejora con evidencia.

Componentes de un sistema de evaluación continua maduro

- **Layer 1 — Evaluación en CI (50-500 casos):** suite offline ejecutada en PRs relevantes con RAGAS o DeepEval; gate de calidad con umbrales configurables que bloquea merges con degradación; costo mínimo viable del sistema de evaluación.
- **Layer 2 — Evaluación online por muestreo (1-5% del tráfico):** job asíncrono con LLM-as-judge sobre muestra del tráfico real; métricas publicadas al dashboard de monitoreo con alertas configuradas; detecta fallos que el dataset offline no captura.
- **Layer 3 — Feedback loop automático:** pipeline que mueve casos con evaluación negativa —score bajo, feedback negativo del usuario, regeneraciones— al dataset offline; el dataset crece con los fallos reales del sistema en lugar de quedarse estático.
- **Layer 4 — Evaluación humana periódica (20-50 casos):** revisión mensual por anotadores con conocimiento del dominio; calibra la correlación entre métricas automáticas y juicio humano; actualiza las rúbricas del LLM-as-judge cuando la correlación baja del umbral aceptable.
- **Leaderboard de versiones:** tabla histórica que compara las métricas de calidad de todas las versiones desplegadas —fecha, modelo, versión de prompt, scores— permitiendo ver la evolución de la calidad del sistema a lo largo del tiempo y atribuir mejoras a cambios específicos.

"What gets measured gets managed." — Peter Drucker. En AI Engineering, el sistema que mide su propia calidad de forma continua y sistemática es el que puede mejorar de forma deliberada y demostrable, en lugar de mejorar por intuición y sorprenderse cuando la calidad se degrada. El sistema de evaluación continua es el instrumento de navegación que transforma la intuición en evidencia.